

Rapport de réseaux de neurones

Projet de travaux pratiques

Groupe de TP : 4

Numéro d'équipe : 5

Membres :

NOURREDDINE Ahmed Anis

MEZAOUI Damya Tassadit

DRISSI Assil

May 2026

Contents

List of Figures

List of Tables

Liste des algorithmes

1	Partie 1 : Construction du modèle de classification MCTNet	1
1.1	Jeux de données	1
1.1.1	Construction	1
1.1.2	Distribution des classes et profils NDVI	2
1.2	Architecture et Entraînement	3
1.2.1	Configuration et Hyperparamètres	3
1.2.2	Modélisation Algorithmique	4
1.2.3	Résultats et Analyse	7
1.2.4	Analyse des Performances et Limites	9
2	Partie 2 : Ajout de covariables environnementales	10
2.1	Enrichissement des jeux de données	10
2.1.1	Sélection et justification des covariables	10
2.1.2	Procédure d'extraction et fusion des données	11
2.1.3	Relations entre covariables et plantations	11
2.2	Modification de l'architecture	14
2.2.1	Réorganisation des données d'entrée	14
2.2.2	Application de l'ECA sur la matrice bandes + climat	14
2.2.3	Encodage positionnel ALPE restreint aux bandes spectrales	15
2.2.4	Fusion tardive des variables statiques	15
2.2.5	Architecture configurable	16
2.3	Étude d'ablation et Résultats	16
3	Partie 3 : Proposition d'architecture améliorée	19
3.1	Problématique : le salt-and-pepper noise dans MCTNet	19
3.1.1	Définition et origine du bruit	19
3.1.2	Mécanismes générateurs du bruit	19
3.2	Axe d'amélioration : régularisation par Batch-KNN	20
3.2.1	Positionnement de la solution	20
3.2.2	Description de l'architecture MCTNet + Batch-KNN	20

3.2.3	Fonctionnement du bloc Batch-KNN	21
3.3	Architecture complète : composantes et modifications	23
3.3.1	Composantes héritées de MCTNet (Parties 1 et 2)	23
3.3.2	Modifications apportées en Partie 3	23
3.4	Résultats et analyse	23
3.4.1	Configurations évaluées	23
3.4.2	Résultats quantitatifs	25
3.4.3	Analyse	25

Bibliography

List of Figures

1.1	Évolution temporelle du NDVI moyen pour chaque classe de culture.	3
1.2	Graphes de perte et précision du modèle sur le jeu de données de base d'Arkansas	8
1.3	Graphes de perte et précision du modèle sur le jeu de données de base de Californie	8
1.4	Matrices de confusion du modèle suite à l'entraînement sur les jeux de données de base d'Arkansas et Californie	9
2.1	Distribution des covariables environnementales continues (météorologie, pH, carbone organique et élévation) en fonction des types de cultures en Arkansas.	12
2.2	Distribution des covariables environnementales continues (météorologie, pH, carbone organique et élévation) en fonction des types de cultures en Californie.	12
2.3	Répartition proportionnelle des covariables catégorielles (texture du sol et type de relief) par classe de culture en Arkansas.	13
2.4	Répartition proportionnelle des covariables catégorielles (texture du sol et type de relief) par classe de culture en Californie.	13
2.5	Courbes d'apprentissage (Fonction de perte et Accuracy globale) pour la configuration : climat uniquement d'Arkansas.	17
2.6	Courbes d'apprentissage (Fonction de perte et Accuracy globale) pour la configuration : climat uniquement de Californie.	17
2.7	Matrices de confusion suite a la configuration climat uniquement avec les deux jeux de données	17
3.1	Courbes d'évaluation de la perte / précision pour MCTNet + Batch-KNN (Version 2) sur Arkansas.	24
3.2	Courbes d'évaluation de la perte / précision pour MCTNet + Batch-KNN (Version 2) sur Californie	24
3.3	Matrices de confusion pour MCTNet + Batch-KNN (Version 2). . . .	25

List of Tables

1.1	Distribution des échantillons pour le jeu de données de la Californie. .	2
1.2	Distribution des échantillons pour le jeu de données de l'Arkansas. . .	2
1.3	Performances du modèle MCTNet (meilleurs runs) sur les jeux de test Arkansas et Californie	8
2.1	Configurations de covariables testées dans l'étude d'ablation.	16
2.2	Résultats de l'étude d'ablation pour l'Arkansas.	16
2.3	Résultats de l'étude d'ablation pour la Californie.	17
3.1	Description des trois configurations d'entrée évaluées	23
3.2	Comparaison des métriques pour les trois versions sur Arkansas et Californie (meilleurs résultats sur 10 runs)	25

List of Algorithms

1	Chargement et division du jeu de données	4
2	Module ECA — Attention inter-canaux	5
3	Module ALPE	5
4	CNNSubModule	6
5	TransformerSubModule	6
6	CTFusionBlock	6
7	MCTNet	7
8	Boucle d'entraînement avec Early Stopping	7
9	Batch-KNN Smoothing	21

Partie 1 : Construction du modèle de classification MCTNet

Notre but durant cette partie est de suivre la méthodologie décrite dans l'étude "*A lightweight CNN-Transformer network for pixel-based crop mapping using time-series Sentinel-2 imagery*" par Wang et al. (2024). Pour ce faire, nous devons reproduire fidèlement le modèle MCTNet décrit ainsi que le jeu de données produit.

1.1 Jeux de données

1.1.1 Construction

La constitution du jeu de données a été réalisée entièrement sur Google Earth Engine (GEE), en suivant la méthodologie décrite dans le papier de référence, avec une adaptation majeure : plutôt que de sélectionner deux zones géographiques restreintes par État, l'intégralité des territoires de l'Arkansas et de la Californie a été retenue comme zone d'étude.

La procédure de création a suivi les étapes suivantes :

1. **Données de référence et masquage spatial** : Les étiquettes de classes ont été extraites à partir de la base de données Cropland Data Layer (CDL) 2021, filtrée à un seuil de confiance de 95 % afin de ne conserver que les pixels les mieux caractérisés. Un masque de zones non agricoles a été appliqué en amont en utilisant ESA WorldCover 2021, limitant ainsi l'échantillonnage aux champs cultivés.
2. **Prétraitement des images satellitaires** : L'ensemble des images Sentinel-2 de Niveau-2A disponibles sur l'année 2021 a été collecté. Après l'élimination des pixels affectés par les nuages, dix bandes spectrales pertinentes ont été retenues : Bleu (B2), Vert (B3), Rouge (B4), Red Edge 1–3 (B5, B6, B7), NIR (B8), Red Edge 4 (B8A) et SWIR 1–2 (B11, B12). Pour construire les séries temporelles, la valeur médiane des observations valides a été calculée sur des intervalles de 10 jours. Ce traitement génère une séquence de 36 pas de temps couvrant l'année 2021 entière pour chaque bande. Lorsqu'aucune observation n'était disponible sur une fenêtre donnée, la valeur manquante a été fixée à 0. Le masque de présence/absence de données valides a par ailleurs été calculé pour chaque pas de temps (1 pour observation valide, et 0 pour non valide).

3. **Échantillonnage et extraction des caractéristiques :** Enfin, 10 000 points par État ont été choisis de manière aléatoire à partir de nos zones préalablement masquées. Pour chacun de ces points, les coordonnées géographiques, l'étiquette de la culture (issue de la CDL) et les valeurs des séries temporelles Sentinel-2 ont été extraites.

Au total, chaque échantillon de ce jeu de données possède une étiquette, des coordonnées, 360 informations spectrales (10 bandes $\times$ 36 pas de temps) et 36 masques de validation (un par pas de temps).

1.1.2 Distribution des classes et profils NDVI

Le tableau 1.1 et le tableau 1.2 présentent la répartition exacte des échantillons extraits pour les deux zones d'étude.

Classe	Total	Entraînement	Validation	Test
[0] Grapes	2008	240	60	1708
[1] Rice	1856	240	60	1556
[2] Alfalfa	985	240	60	685
[3] Almonds	808	240	60	508
[4] Pistachios	585	240	60	285
[5] Others	3758	240	60	3458
Total Global	10 000	1440	360	8200

Table 1.1: Distribution des échantillons pour le jeu de données de la Californie.

Classe	Total	Entraînement	Validation	Test
[0] Soybeans	4704	240	60	4404
[1] Rice	2488	240	60	2188
[2] Corn	1392	240	60	1092
[3] Cotton	806	240	60	506
[4] Others	610	240	60	310
Total Global	10 000	1200	300	8500

Table 1.2: Distribution des échantillons pour le jeu de données de l'Arkansas.

Afin d'illustrer la pertinence des séries temporelles acquises, nous avons calculé l'Indice de Végétation par Différence Normalisée (NDVI) pour les différentes cultures. L'évolution au cours des 36 pas de temps est présentée ci-dessous.

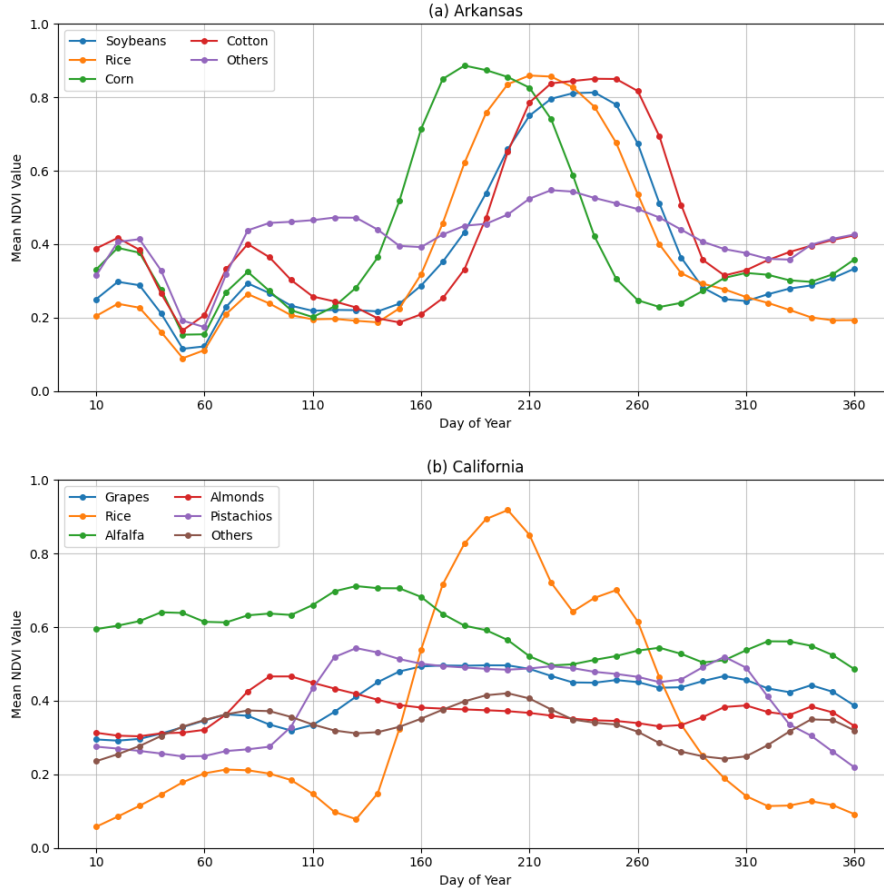


Figure 1.1: Évolution temporelle du NDVI moyen pour chaque classe de culture.

1.2 Architecture et Entraînement

Cette section présente l'implémentation du modèle **MCTNet** (*Multi-scale Convolutional and Transformer Network*), reproduit à partir de l'étude "*A lightweight CNN-Transformer network for pixel-based crop mapping using time-series Sentinel-2 imagery*". Nous y décrivons les différents modules constituant l'architecture, les hyperparamètres utilisés lors de l'entraînement ainsi que les performances obtenues sur les deux jeux de données étudiés.

1.2.1 Configuration et Hyperparamètres

Le modèle a été entraîné sur GPU via **Kaggle** en utilisant le framework **PyTorch**. Les hyperparamètres retenus afin d'assurer la convergence et la robustesse de l'apprentissage sont les suivants :

1. Optimiseur : Adam avec un taux d'apprentissage (lr) de 0,001.
2. Fonction de perte : Cross-Entropy Loss.
3. Taille de lot (*Batch Size*) : 32.
4. Stratégie d'entraînement : 200 époques maximum avec un mécanisme d'*Early Stopping* (*patience* = 20) basé sur la perte de validation, afin de prévenir le surapprentissage.

1.2.2 Modélisation Algorithmique

Conformément à l’architecture MCTNet, les modules **ECA**, **ALPE** et **CTFusion** ont été implémentés pour capturer respectivement les dépendances spectrales (inter-canaux), spatiales (positionnelles) et spatio-temporelles (fusion CNN-Transformer). Les pseudo-algorithmes de chaque module sont présentés ci-dessous dans leur ordre d’exécution au sein du modèle.

1. Préparation des données

Le premier algorithme décrit la structuration des données brutes en tenseur 3D et leur division stratifiée en ensembles d’entraînement, de validation et de test.

Algorithm 1 Chargement et division du jeu de données

```
1: Initialiser le chemin et les noms des fichiers des datasets
2: target_bands  $\leftarrow$  noms des bandes spectrales
3: my_labels  $\leftarrow$  attribuer à chaque valeur unique de nos labels un nom correspondant à la plantation
4:  $X\_list, y\_list, M\_list \leftarrow [], [], []$ 
5: pour chaque fichier  $f$  dans FILES faire
6:   Vérifier l’existence du fichier
7:   Ouvrir le CSV du dataset  $\Rightarrow df$ 
8:    $temp\_X \leftarrow []$ 
9:   pour chaque bande  $b$  dans target_bands faire
10:    Rassembler toutes les valeurs temporelles de  $b$  dans le dataset et les ajouter
11:    à la liste  $temp\_X$ :  $temp\_X.append(\text{valeurs temporelles de } b)$ 
12:   fin pour
13:    $X\_3d \leftarrow$  empiler  $temp\_X$  en un tenseur 3D de forme  $(B, 36, 10)$ 
14:    $y\_names \leftarrow$  convertir les labels numériques en noms de cultures via my_labels
15:    $M\_2d \leftarrow$  extraire le masque de validité nuageux de forme  $(B, 36)$ 
16:    $X\_list.append(X\_3d)$ 
17:    $y\_list.append(y\_names)$ 
18:    $M\_list.append(M\_2d)$ 
19: fin pour
20: Fusionner  $X\_list, y\_list$  et  $M\_list$  en tableaux uniques  $X, y\_raw$  et  $M$ 
21: Fixer la graine aléatoire
22: pour chaque classe  $c$  parmi les classes uniques de  $y$  faire
23:   Récupérer et mélanger aléatoirement les indices de  $c$ 
24:   Prendre les 240 premiers  $\rightarrow$  ensemble d’entraînement
25:   Prendre les 60 suivants  $\rightarrow$  ensemble de validation
26:   Garder le reste  $\rightarrow$  ensemble de test
27: fin pour
28: Créer train_loader, val_loader et test_loader avec BATCH_SIZE = 32
```

2. Modules de l’architecture

2.1 Module ECA Le module **ECA** (*Efficient Channel Attention*) applique un mécanisme d’attention légère inter-canaux sans réduction de dimensionnalité.

Il pondère chaque canal spectral selon son importance relative via une convolution 1D locale.

Algorithm 2 Module ECA — Attention inter-canaux

Entrée : tenseur x de forme (B, C, T) , où C est le nombre de canaux et T représente le temps

Sortie : tenseur x repondéré

- 1: $y \leftarrow$ Appliquer un global pooling moyen sur x de sorte à réduire la dimension temporelle à $(B, C, 1)$: $\text{AdaptiveMaxPool1D}_{\text{output}=1}(y)$
 - 2: Transposer y en $(B, 1, C)$
 - 3: $y \leftarrow$ Appliquer une convolution 1D entre bandes voisines $\text{Conv1D}_1(y)$
 - 4: Transposer y en $(B, C, 1)$
 - 5: $y \leftarrow$ Appliquer la fonction d'activation *sigmoid* $\sigma(y)$
 - 6: **retourner** $x \odot \sigma(y)$
-

2.2 Module ALPE — Encodage Positionnel Adaptatif Le module **ALPE** (*Adaptive Learnable Positional Encoding*) génère un encodage positionnel sinusoïdal masqué par la couverture nuageuse, puis l'affine via une convolution et le module **ECA**.

Algorithm 3 Module ALPE

Entrée : tenseur x de forme $(B, 36, 10)$, $mask$ booléen $(B, 10)$

Sortie : pe de forme $(B, 36, 10)$

Initialisation

- 1: $pe \leftarrow$ Générer la matrice d'encodage positionnel sinusoïdal de forme $(B, 36, 10)$
 - Passage forward*
 - 2: Étendre pe pour correspondre à la taille du batch B
 - 3: Étendre $mask$ en $(B, 36, 1)$
 - 4: Appliquer le masque de validité $pe \leftarrow pe \odot mask$
 - 5: $pe \leftarrow \text{Conv1D}_{3 \times 3}(\text{Transposé de } (pe))$
 - 6: $pe \leftarrow \text{ECA}(pe)$
 - 7: Transposer pe en $(B, 36, 10)$
 - 8: **retourner** pe
-

2.3 Bloc CTFusion Le bloc **CTFusion** fusionne en parallèle une branche **CNN** (captures locales) et une branche **Transformer** (dépendances à long terme). Chaque branche est détaillée ci-dessous avant la présentation du bloc de fusion.

Algorithm 4 CNNSubModule

Entrée : tenseur x de forme $(B, 36, 10)$

Sortie : h

- 1: Transposer x en $(B, 10, 36)$
 - 2: $h \leftarrow x$
 - 3: $residual \leftarrow \text{Conv1D}_{1 \times 1}(h)$
 - 4: $h \leftarrow \text{Conv1D}_{3 \times 3}(h)$
 - 5: $h \leftarrow \text{BatchNorm1D}(h)$
 - 6: $h \leftarrow \text{Conv1D}_{3 \times 3}(h)$
 - 7: $h \leftarrow \text{BatchNorm1D}(h)$
 - 8: Additionner la sortie des convolutions au raccourci résiduel $h \leftarrow h + residual$
 - 9: $h \leftarrow$ Appliquer la fonction d'activation ReLU : $\text{ReLU}(h)$
 - 10: **retourner** h
-

Algorithm 5 TransformerSubModule

Entrée : tenseur x , encodage de position $pe = \text{None}$

Sortie : x_3

- 1: $x_{pe} = x + \begin{cases} pe & \text{si } pe \neq \emptyset \\ 0 & \text{sinon} \end{cases}$
 - 2: Calculer l'auto-attention multi-têtes sur x_{pe} :
 $attn \leftarrow \text{MultiHeadAttention}(x_{pe}, x_{pe}, x_{pe})$
 - 3: $x_2 \leftarrow attn + x_{pe}$
 - 4: $x_2 \leftarrow \text{LayerNorm}(x_2)$
 - 5: $ffn_out \leftarrow \text{Linear}_1(x_2)$
 - 6: $ffn_out \leftarrow \text{ReLU}(ffn_out)$
 - 7: $ffn_out \leftarrow \text{Linear}_2(ffn_out)$
 - 8: $ffn_out \leftarrow ffn_out + x_2$
 - 9: $x_3 \leftarrow \text{LayerNorm}(x_3 + x_2)$
 - 10: **retourner** x_3
-

Algorithm 6 CTFusionBlock

Entrée : tenseur x , encodage de position $pe = \text{None}$

Sortie : $fused$

- 1: $t_{out} \leftarrow \text{TransformerSubModule}(x, pe)$
 - 2: $c_{out} \leftarrow \text{CNNSubModule}(x)$
 - 3: Transposer t_{out}
 - 4: $fused \leftarrow \text{Concat}(c_{out}, t_{out})_{\text{dim}=1}$
 - 5: $fused \leftarrow \text{Conv1D}_1(fused)$
 - 6: $fused \leftarrow$ Appliquer un Max pooling d'une dimension avec fenêtre de deux sur $fused$: $\text{MaxPool1D}(fused)$
 - 7: Transposer $fused$
 - 8: **retourner** $fused$
-

3. Architecture Globale MCTNet

L'algorithme ci-dessous décrit l'architecture complète du modèle **MCTNet**, composé d'un encodeur positionnel **ALPE** suivi de trois blocs **CTFusion** en cascade.

À chaque stage, le nombre de canaux est doublé et le nombre de pas de temps est divisé par deux ($36 \rightarrow 18 \rightarrow 9 \rightarrow 4$), permettant une représentation hiérarchique croissante.

Algorithm 7 MCTNet

Entrée : tenseur x , $mask$ booléen
Sortie : $\hat{y}$
Initialisation
1: Initialisation des 3 blocs CTFusion ; à chaque stage le nombre de pas de temps est divisé par deux et le nombre de canaux est doublé :
 $stage1 \leftarrow \text{CTFusionBlock}(in = 10, out = 20, n_head = 5)$
 $stage2 \leftarrow \text{CTFusionBlock}(in = 20, out = 40, n_head = 5)$
 $stage3 \leftarrow \text{CTFusionBlock}(in = 40, out = 80, n_head = 5)$
2: Initialisation du classifieur final :
 $classifier \leftarrow \text{Linear}(80, n_classes)$
Passage forward
3: $pe \leftarrow \text{ALPE}(x, mask)$
4: $x \leftarrow \text{CTFusionBlock}_1(x, pe)$ (PE injecté uniquement ici)
5: $x \leftarrow \text{CTFusionBlock}_2(x)$
6: $x \leftarrow \text{CTFusionBlock}_3(x)$
7: $x \leftarrow$ Appliquer un global pooling moyen sur x pour résumer la séquence en 1 seul vecteur : $\text{AdaptiveMaxPool1D}_{output=1}(x)$
8: Projeter le vecteur de caractéristiques vers les scores de chaque classe de culture :
 $\hat{y} \leftarrow classifier(x)$
9: **retourner** $\hat{y}$

4. Boucle d'Entraînement avec Early Stopping

Algorithm 8 Boucle d'entraînement avec Early Stopping

Initialisation des hyperparamètres
1: Définir les hyperparamètres :
 $EPOCHS = 200, lr = 0.001, PATIENCE = 20, BATCH_SIZE = 32$
2: Initialiser l'optimiseur :
 $optimizer \leftarrow \text{Adam}(model.params, lr = 0.001)$
3: Initialiser la fonction de perte :
 $criterion \leftarrow \text{CrossEntropyLoss}()$
Boucle d'entraînement
4: **pour** $epoch = 1$ **à** $EPOCHS$ **faire**
5: Boucle d'entraînement avec early stopping
6: **fin pour**
7: Restaurer les meilleurs poids sauvegardés

1.2.3 Résultats et Analyse

Les performances du modèle MCTNet ont été évaluées sur deux zones géographiques distinctes. Les métriques présentées ci-dessous correspondent aux meilleurs runs obtenus après stabilisation de l'entraînement, sur les jeux de test respectifs.

Dataset	Overall Accuracy (OA)	Kappa Score	F1-score
Arkansas	0.9671	0.9496	0.9265
California	0.8530	0.8012	0.8177

Table 1.3: Performances du modèle MCTNet (meilleurs runs) sur les jeux de test Arkansas et Californie

Sur le jeu de données de l'Arkansas, le modèle atteint une **Overall Accuracy** de **96,71 %** et un score **Kappa** de **0,9496**, témoignant d'une excellente capacité de classification des cultures. La courbe d'apprentissage (Figure 1.2) montre une convergence rapide et stable, avec une perte d'entraînement descendant à environ **0,01** et une perte de validation se stabilisant autour de **0,2**, un écart caractéristique d'un surapprentissage modéré mais contrôlé par l'Early Stopping.

Sur le jeu de données de Californie, les performances du modèle sont plus modestes, avec une **Overall Accuracy** de **85,30 %** et un score **Kappa** de **0,8012**. La matrice de confusion (Figure 1.4) met en évidence une difficulté particulière à distinguer entre les classes Amandiers et Pistachiers, leurs signatures spectrales temporelles présentant des comportements très similaires au fil du temps.

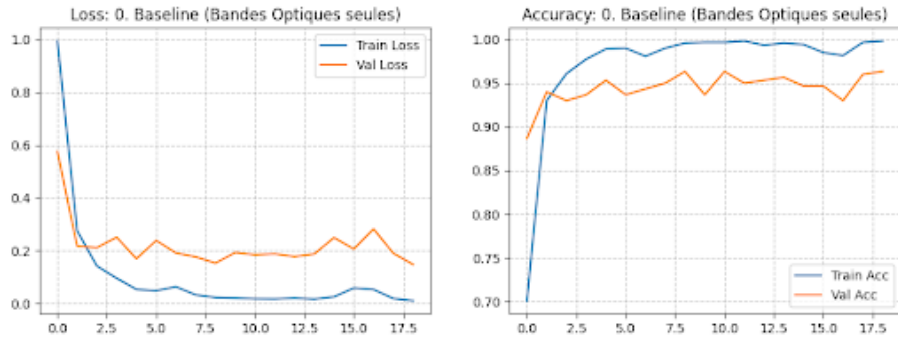


Figure 1.2: Graphes de perte et précision du modèle sur le jeu de données de base d'Arkansas

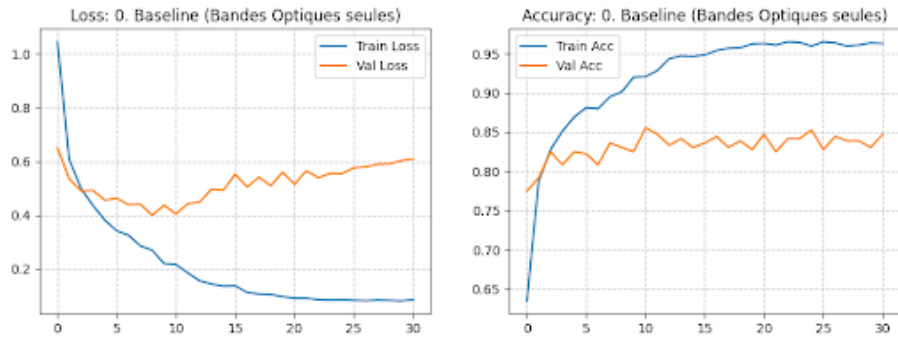


Figure 1.3: Graphes de perte et précision du modèle sur le jeu de données de base de Californie

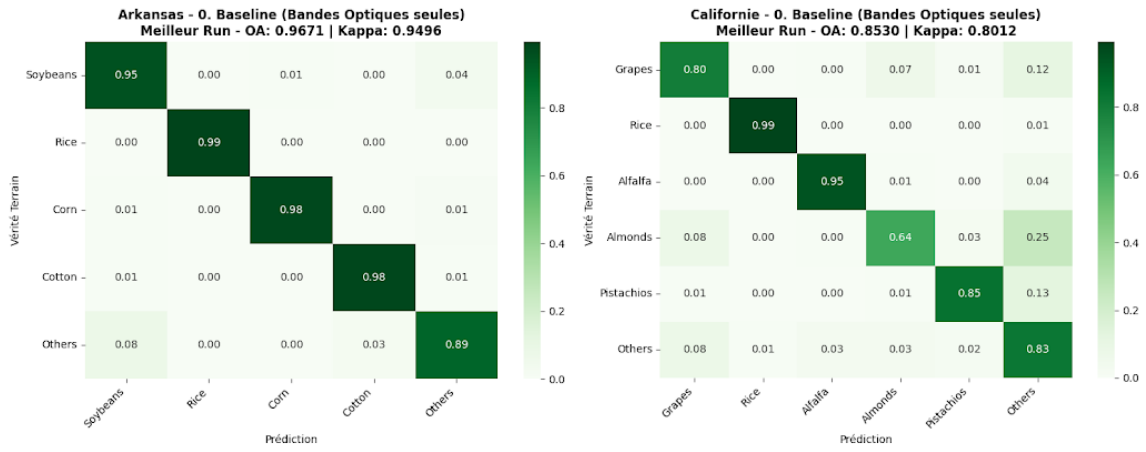


Figure 1.4: Matrices de confusion du modèle suite à l'entraînement sur les jeux de données de base d'Arkansas et Californie

1.2.4 Analyse des Performances et Limites

Deux facteurs principaux expliquent la disparité des performances entre les deux États :

1. **Complexité spectrale des cultures cibles** : L'Arkansas présente des scores très élevés car les cultures cibles (Soja, Riz inondé, Maïs, Coton) possèdent des signatures spectrales temporelles visuellement très distinctes, notamment le Riz, dont la phénologie de repiquage crée un profil NDVI unique. À l'inverse, le jeu de données Californie est principalement composé de vergers tels que les Amandiers, les Pistachiers et les Vignes, dont les comportements spectraux restent très proches au cours du temps, ce qui complique fortement la classification à partir des seules données optiques.
2. **Surapprentissage (Overfitting)** : Malgré l'utilisation de l'*Early Stopping*, les courbes d'apprentissage mettent en évidence un phénomène de surapprentissage important. La perte d'entraînement diminue jusqu'à environ **0,01** pour l'Arkansas et **0,05** pour la Californie, tandis que la perte de validation se stabilise à des valeurs nettement plus élevées, proches de **0,2** et **0,5** respectivement. Ce comportement peut s'expliquer par le caractère fortement *data-hungry* des mécanismes d'attention des Transformers [2], avec seulement **240** échantillons par classe, le modèle tend à mémoriser les micro-variations de l'ensemble d'entraînement plutôt qu'à apprendre des représentations réellement généralisables.

Conclusion Ces deux limites, à savoir la confusion spectrale et le surapprentissage, constituent la motivation principale de la Partie 2, qui explore l'apport des covariables environnementales telles que la climatologie, la pédologie et la topographie afin d'enrichir le pouvoir discriminant du modèle, au-delà de ce que permettent les bandes satellitaires.

Partie 2 : Ajout de covariables environnementales

Pour aider le modèle à différencier des cultures aux signatures spectrales similaires, nous avons émis l'hypothèse que l'ajout d'un contexte environnemental améliorerait la robustesse de la classification. Suite à ce raisonnement, trois types de covariables, décrivant les conditions dans lesquelles poussent les cultures, ont été intégrés aux jeux de données de chaque État : la météorologie, la pédologie (caractéristiques du sol) et la topographie (le relief).

2.1 Enrichissement des jeux de données

2.1.1 Sélection et justification des covariables

1. Les covariables climatiques temporelles (Source : ERA5-Land) :

Contrairement aux données spatiales statiques, le climat évolue tout au long de l'année. Afin d'assurer une parfaite cohérence avec les séries temporelles spectrales (Sentinel-2), les données météorologiques ont été agrégées (moyenne ou somme) sur les mêmes fenêtres temporelles de 10 jours, générant ainsi 36 pas de temps continus pour l'année 2021. Trois variables majeures ont été sélectionnées :

- **La température moyenne (mean air temperature) :** Permet de suivre l'évolution saisonnière et les transitions phénologiques (germination, floraison, maturation) pour chaque type de culture.
- **Les précipitations totales (total precipitation) :** Le cumul des pluies sur des fenêtres de 10 jours permet d'évaluer la quantité d'eau naturellement reçue par les plantes. Cela aide le modèle à faire la différence entre les cultures qui dépendent de la pluie et celles qui ont besoin de beaucoup d'irrigation artificielle (comme le riz ou les amandiers).
- **La température moyenne du point de rosée (mean dewpoint temperature) :** C'est un bon indicateur de l'humidité de l'air. C'est important car certaines cultures (comme la vigne ou le coton) sont très sensibles à l'humidité, ce qui peut favoriser l'apparition de maladies.

Contrairement aux covariables météorologiques, les covariables restantes possèdent des valeurs fixes tout au long de l'année.

2. Les covariables pédologiques (Source : OpenLandMap) :

- **Le pH du sol** : Est un indicateur crucial mesurant l'acidité ou l'alcalinité (entre 0 et 14), qui détermine directement la disponibilité des nutriments pour les plantes. Certains types de plantes prospèrent dans un pH neutre, tandis que d'autres préfèrent un pH alcalin, ce qui donne un bon indice au modèle pour différencier les cultures.
- **Le Carbone Organique** : Est le composant majeur de la matière organique, essentiel à la fertilité, la structure du sol et la régulation climatique.
- **La Texture du sol** : La texture détermine le drainage. Un sol très argileux qui retient l'eau sera privilégié pour la culture du riz, tandis que les cultures fruitières nécessitent des sols plus filtrants.

3. Les covariables topographiques (Sources : ETOPO1 et ALOS Landforms) :

- **L'élévation** : L'altitude modifie les microclimats et la durée de la saison de croissance. Les vergers ou la vigne occupent souvent des coteaux, tandis que le riz ou le soja s'étendent quasi exclusivement dans les plaines basses.
- **Le type de relief (Landform)** : La morphologie du terrain dicte la productivité, le type de cultures et les techniques utilisées. Il influence directement l'érosion des sols, l'irrigation, la mécanisation, et le microclimat. En plaine, la culture est facilitée, tandis qu'en zone pentue, l'élevage et la viticulture prédominent.

2.1.2 Procédure d'extraction et fusion des données

Afin de fusionner ces nouvelles informations avec les données optiques existantes, un processus d'extraction a été développé sur Google Earth Engine (GEE).

En utilisant les coordonnées géographiques des points préalablement extraits, les valeurs moyennes (ou cumulées pour la pluie) des variables climatiques ont été calculées pour chacune des 36 fenêtres de 10 jours utilisées pour générer les composites Sentinel-2. Ainsi, chaque pas de temps optique possède son pendant météorologique exact. Également, les variables statiques (le profil du sol et topographie) ont été échantillonnées spatialement sur ces mêmes emplacements géographiques.

Au final, chaque échantillon de cette nouvelle base de données est décrit par une matrice combinant les séries temporelles spectrales d'origine, les séries temporelles climatiques (3 variables $\times$ 36 pas de temps) et les constantes environnementales fixes (sol et relief).

2.1.3 Relations entre covariables et plantations

Avant d'entraîner notre modèle, une analyse exploratoire a été menée pour visualiser la distribution des types de cultures en fonction des nouvelles covariables environnementales. L'objectif est de vérifier si ces variables pourront pallier la faille de robustesse du modèle.

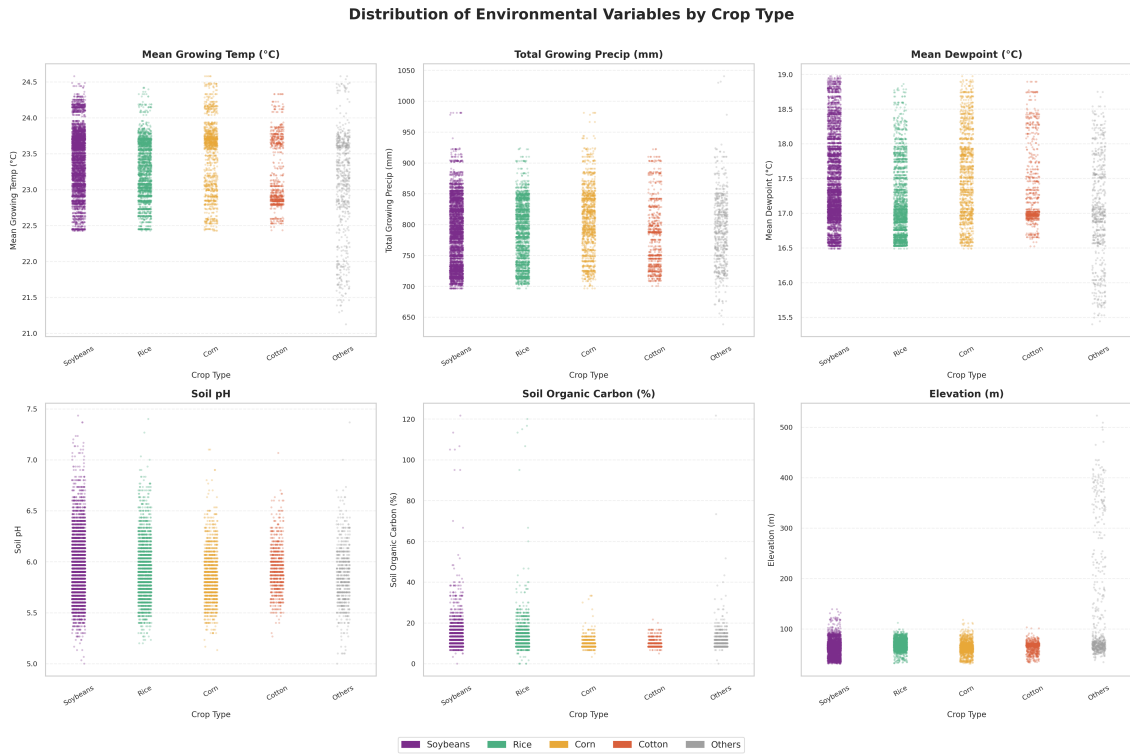


Figure 2.1: Distribution des covariables environnementales continues (météorologie, pH, carbone organique et élévation) en fonction des types de cultures en Arkansas.

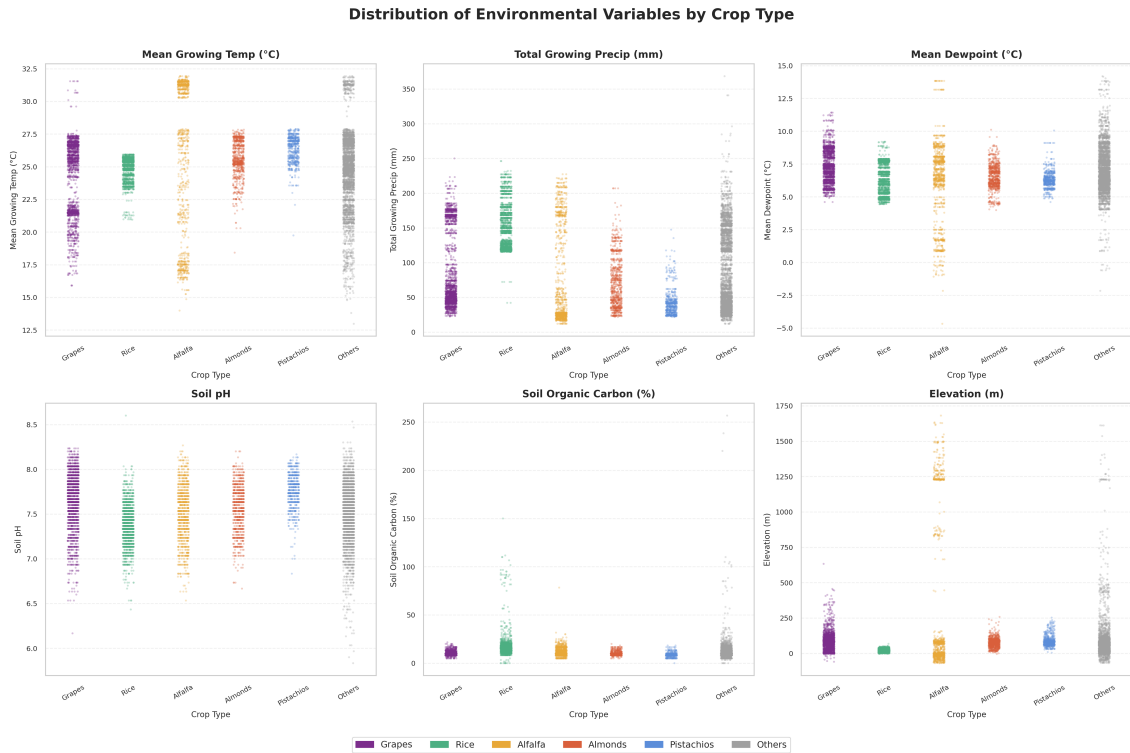


Figure 2.2: Distribution des covariables environnementales continues (météorologie, pH, carbone organique et élévation) en fonction des types de cultures en Californie.

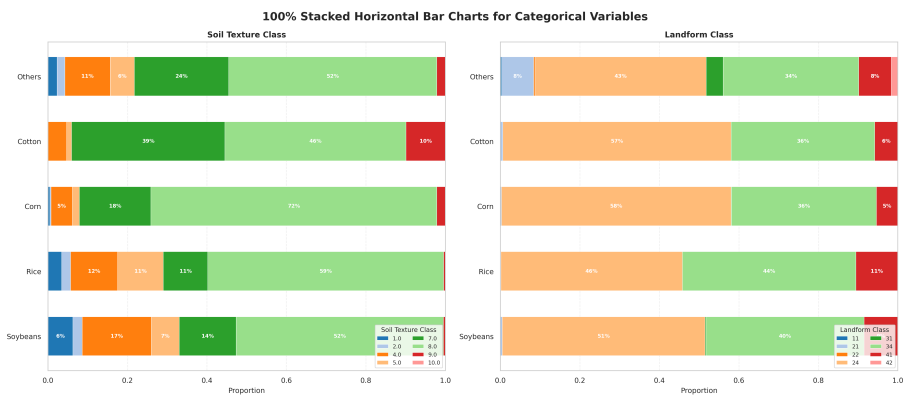


Figure 2.3: Répartition proportionnelle des covariables catégorielles (texture du sol et type de relief) par classe de culture en Arkansas.

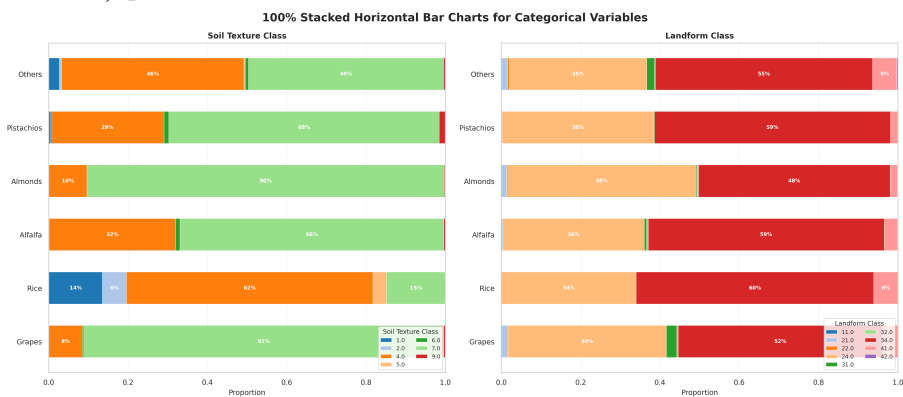


Figure 2.4: Répartition proportionnelle des covariables catégorielles (texture du sol et type de relief) par classe de culture en Californie.

Voici ce que chaque type de variable apporte comme connaissance :

- **La topographie (Élévation et Landform) :**

L'élévation s'impose comme l'une des variables les plus séparatrices en Californie (Figure 2.2).

On observe que le Riz y est cultivé uniquement à très basse altitude avec une variance quasi nulle. À l'inverse, la Luzerne (Alfalfa) présente une distribution bimodale, avec une part importante des parcelles situées à plus de 1000 mètres d'altitude.

En ce qui concerne l'Arkansas (Figure 2.1), bien que les cultures cibles (Soja, Riz, Maïs, Coton) partagent toutes des altitudes basses (< 150m), l'élévation permet de distinguer nettement la classe "Autres" (Others) qui s'étend jusqu'à 500 mètres.

- **La pédologie (Texture et Carbone Organique) :**

Les graphiques catégoriels révèlent que certaines cultures exigent des types de sols très spécifiques.

En Californie (Figure 2.4), la texture du sol isole parfaitement le Riz. Tandis que la majorité des cultures dominent massivement sur la texture de classe 7.0, le Riz se démarque totalement avec une majorité de sols de classe 4.0 et 1.0. Concernant le Carbone Organique, on note dans les deux États que les parcelles de Riz présentent une variance beaucoup plus large, s'étendant vers des valeurs de carbone nettement supérieures aux autres cultures, ce qui constitue une signature pédologique forte.

- **La météorologie :**

Bien que les variables climatiques montrent un chevauchement logique à l'échelle d'un État, elles révèlent des "zones de confort" spécifiques (visibles sous forme d'amas de points).

En Californie, le Riz montre une tolérance climatique très stricte, tandis que d'autres cultures comme la Luzerne ou les Vignes démontrent une grande plasticité face aux variations de température et d'humidité.

Conclusion de l'analyse exploratoire : Ces visualisations confirment notre hypothèse initiale. Les cultures spectralement proches (comme les différents arbres fruitiers) ou difficiles à isoler possèdent en réalité des niches environnementales distinctes (altitudes différentes, exigences de sols opposées). L'ajout de ces covariables offre donc au modèle MCTNet des "raccourcis" logiques pour distinguer les pixels lors de la classification.

2.2 Modification de l'architecture

Afin d'intégrer les covariables environnementales au sein du réseau MCTNet, l'architecture originale a été légèrement adaptée, tout en conservant l'ensemble de la structure CNN-Transformer initiale. Les modifications apportées se concentrent principalement sur quatre aspects distincts, présentés ci-dessous.

2.2.1 Réorganisation des données d'entrée

Dans le modèle de référence, chaque échantillon est entièrement décrit par une unique matrice temporelle de forme $X \in \mathbb{R}^{36 \times 10}$, regroupant les 10 bandes spectrales Sentinel-2 sur les 36 pas de temps.

Avec l'introduction des covariables, les données d'entrée sont désormais organisées en trois groupes distincts : les bandes spectrales Sentinel-2, les variables climatiques temporelles et les variables statiques environnementales. Puisque les variables climatiques partagent la même résolution temporelle que les images satellitaires, elles sont directement concaténées aux bandes spectrales. La matrice d'entrée principale devient alors :

$$X_{temp} \in \mathbb{R}^{36 \times 13}$$

Les variables statiques (pédologie et topographie), dont les valeurs sont fixes dans le temps, sont conservées dans une matrice indépendante et injectées uniquement en fin de réseau, au moment de la classification.

2.2.2 Application de l'ECA sur la matrice bandes + climat

Une différence clé par rapport au modèle de référence réside dans l'application du module ECA. Alors que dans la version originale celui-ci opère uniquement à l'intérieur de chaque bloc CTFusion, un ECA supplémentaire est ici appliqué en amont de l'architecture, directement sur la matrice d'entrée complète X_{temp} :

- $X_{weighted} \leftarrow \text{ECA}(\text{Transposé de } X_{temp})$

- $X_{weighted} \leftarrow \text{Transposé de } X_{weighted}$
- $x \leftarrow \text{Linear}(X_{weighted})$

Ainsi, le réseau apprend dès l'entrée à pondérer l'importance relative de chaque canal (spectral ou climatique) avant de les traiter conjointement dans les blocs CTFusion.

2.2.3 Encodage positionnel ALPE restreint aux bandes spectrales

Contrairement à ce que l'on pourrait attendre, l'encodage positionnel adaptatif (ALPE) n'est pas calculé sur la matrice complète X_{temp} , mais uniquement sur les bandes spectrales Sentinel-2 :

- $bands_only \leftarrow x_temp[:, :, 0 : n_bands = 10]$
- $pe \leftarrow \text{ALPE}(bands_only, mask)$
- $pe \leftarrow \text{Linear}(pe)$

Ce choix s'explique par le fait que l'ALPE s'appuie sur le masque de validité nuageuse pour pondérer l'encodage positionnel sinusoïdal, un mécanisme propre aux observations optiques. Les variables climatiques, issues d'ERA5-Land, ne souffrent d'aucune occultation nuageuse et n'ont donc pas besoin de ce traitement. L'encodage positionnel ainsi obtenu est ensuite injecté dans le premier bloc CTFusion, de la même manière que dans le modèle de base.

2.2.4 Fusion tardive des variables statiques

Après le Global Max Pooling, le vecteur de caractéristiques appris par MCTNet est concaténé aux covariables statiques avant d'être transmis au classifieur :

```

si  $x_{static}$  non vide alors
     $x \leftarrow \text{Concat}(x, x_{static})$ 
fin si
 $x \leftarrow \text{Linear}(x)$ 
 $\hat{y} \leftarrow \text{classifier}(x)$ 

```

Cette stratégie présente deux avantages complémentaires :

- Les variables climatiques, étant temporelles, bénéficient pleinement des mécanismes d'attention du Transformer au sein des blocs CTFusion ;
- Les variables statiques (sol, relief), dont la nature non-temporelle les rendrait peu exploitables dans un traitement séquentiel, sont injectées en fin de réseau, là où elles enrichissent directement l'espace de décision du classifieur.

Le processus de classification complet devient ainsi : $\text{classifier} \leftarrow x \oplus x_{static}$

2.2.5 Architecture configurable

Afin de faciliter l'étude d'ablation, le programme a été conçu de façon modulaire. Chaque famille de covariables (climat, pédologie, topographie) peut être activée, désactivée ou combinée dynamiquement via la configuration, sans qu'aucune modification du code du réseau ne soit nécessaire. Il suffit par exemple de vider les listes correspondantes pour exclure une famille de variables :

- $time_varying_bases \leftarrow [mean_dewpoint_t, total_precip_t, mean_temp_t]$
- $static_covars \leftarrow [soil_ph, soil_organic_carbon, \dots]$

Les sept configurations testées dans l'étude d'ablation sont ainsi générées à partir d'une unique architecture paramétrable :

Configuration	Bandes Sentinel-2	Climat	Sol	Topographie
Modèle de référence	✓	×	×	×
Climat uniquement	✓	✓	×	×
Sol uniquement	✓	×	✓	×
Topographie uniquement	✓	×	×	✓
Climat + Sol	✓	✓	✓	×
Climat + Topographie	✓	✓	×	✓
All-In	✓	✓	✓	✓

Table 2.1: Configurations de covariables testées dans l'étude d'ablation.

2.3 Étude d'ablation et Résultats

Afin de mesurer l'impact réel de ces nouvelles variables, nous avons mené une étude d'ablation systématique comparant 7 combinaisons différentes de covariables face à la Baseline. Pour garantir la robustesse de l'analyse, chaque configuration a fait l'objet de 20 entraînements complets, permettant d'en extraire la moyenne et l'écart-type.

Configuration	Accuracy (OA)		Kappa		F1-Score	
	Moy. $\pm$ Std	Max	Moy. $\pm$ Std	Max	Moy. $\pm$ Std	Max
0. Baseline (Optique seul)	0.9557 $\pm$ 0.0102	0.9671	0.9331 $\pm$ 0.0148	0.9496	0.9123 $\pm$ 0.0140	0.9265
1. Sol uniquement	0.9525 $\pm$ 0.0076	0.9612	0.9281 $\pm$ 0.0110	0.9408	0.9093 $\pm$ 0.0107	0.9228
2. Topographie uniquement	0.9539 $\pm$ 0.0108	0.9681	0.9304 $\pm$ 0.0156	0.9511	0.9096 $\pm$ 0.0151	0.9280
3. Climat uniquement	0.9610 $\pm$ 0.0058	0.9709	0.9407 $\pm$ 0.0087	0.9554	0.9212 $\pm$ 0.0087	0.9374
4. Climat + Sol	0.9524 $\pm$ 0.0061	0.9668	0.9279 $\pm$ 0.0090	0.9492	0.9094 $\pm$ 0.0104	0.9337
5. Climat + Topographie	0.9594 $\pm$ 0.0057	0.9684	0.9382 $\pm$ 0.0086	0.9516	0.9192 $\pm$ 0.0089	0.9295
6. Sol + Topographie	0.9503 $\pm$ 0.0106	0.9606	0.9248 $\pm$ 0.0152	0.9398	0.9058 $\pm$ 0.0156	0.9194
7. All-in (Toutes variables)	0.9456 $\pm$ 0.0112	0.9658	0.9181 $\pm$ 0.0164	0.9477	0.9006 $\pm$ 0.0166	0.9325

Table 2.2: Résultats de l'étude d'ablation pour l'Arkansas.

Configuration	Accuracy (OA)		Kappa		F1-Score	
	Moy. $\pm$ Std	Max	Moy. $\pm$ Std	Max	Moy. $\pm$ Std	Max
0. Baseline (Optique seul)	0.8213 $\pm$ 0.0120	0.8530	0.7655 $\pm$ 0.0139	0.8012	0.7784 $\pm$ 0.0174	0.8177
1. Sol uniquement	0.8204 $\pm$ 0.0168	0.8435	0.7647 $\pm$ 0.0198	0.7932	0.7826 $\pm$ 0.0175	0.8061
2. Topographie uniquement	0.8163 $\pm$ 0.0124	0.8349	0.7599 $\pm$ 0.0149	0.7819	0.7712 $\pm$ 0.0129	0.7825
3. Climat uniquement	0.8379 $\pm$ 0.0133	0.8656	0.7860 $\pm$ 0.0161	0.8195	0.8053 $\pm$ 0.0144	0.8371
4. Climat + Sol	0.8343 $\pm$ 0.0174	0.8673	0.7815 $\pm$ 0.0203	0.8212	0.7976 $\pm$ 0.0183	0.8278
5. Climat + Topographie	0.8265 $\pm$ 0.0176	0.8483	0.7719 $\pm$ 0.0207	0.7986	0.7924 $\pm$ 0.0212	0.8230
6. Sol + Topographie	0.8210 $\pm$ 0.0124	0.8407	0.7654 $\pm$ 0.0145	0.7886	0.7824 $\pm$ 0.0150	0.8128
7. All-in (Toutes variables)	0.8257 $\pm$ 0.0179	0.8590	0.7712 $\pm$ 0.0210	0.8105	0.7920 $\pm$ 0.0153	0.8184

Table 2.3: Résultats de l'étude d'ablation pour la Californie.

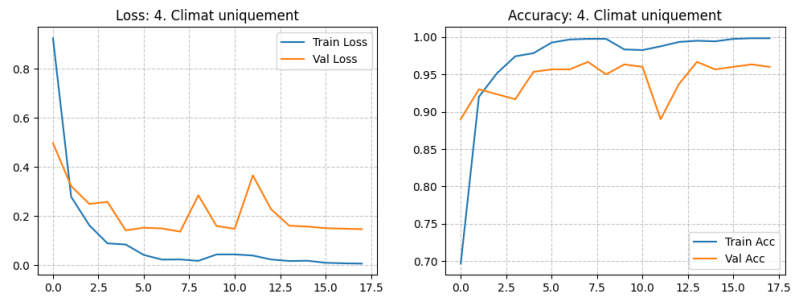


Figure 2.5: Courbes d'apprentissage (Fonction de perte et Accuracy globale) pour la configuration : climat uniquement d'Arkansas.

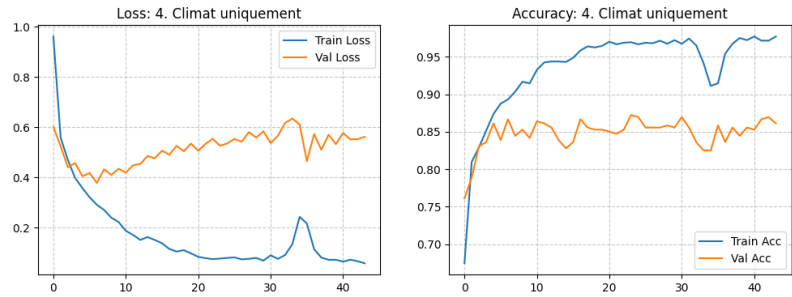


Figure 2.6: Courbes d'apprentissage (Fonction de perte et Accuracy globale) pour la configuration : climat uniquement de Californie.

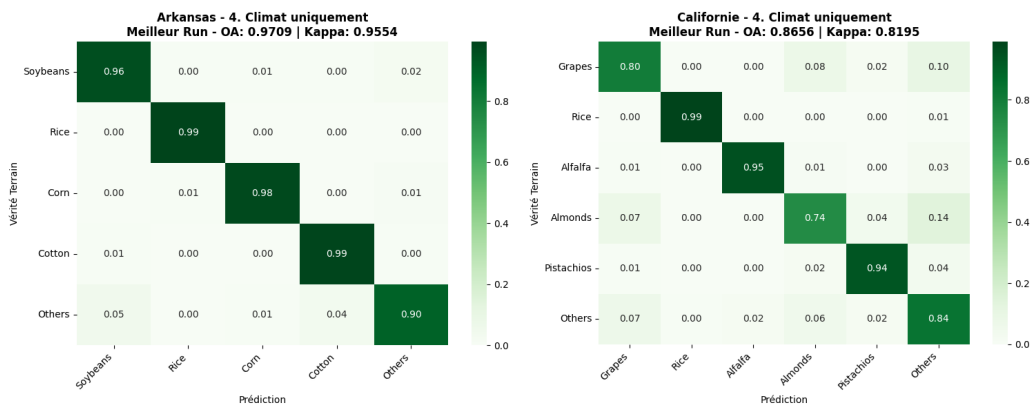


Figure 2.7: Matrices de confusion suite a la configuration climat uniquement avec les deux jeux de données

L'analyse des métriques de performance sur la Californie et l'Arkansas révèle trois conclusions fondamentales sur le comportement du modèle MCTNet face aux données environnementales :

1. L'impact des valeurs temporelles du climat :
Que ce soit sur le jeu de données complexe de Californie (OA : 0.8379) ou sur celui de l'Arkansas (OA : 0.9610), la configuration "Climat uniquement" donne les résultats les plus concluants. Agronomiquement, les variables météorologiques (températures, précipitations) expliquent les variations phénologiques (floraison, maturation) observées par les bandes spectrales. Le réseau Transformer, conçu pour l'analyse temporelle, entre en parfaite synergie avec ces données dynamiques.
2. L'échec du "All-In" :
Intuitivement, l'injection de l'intégralité des variables disponibles aurait dû maximiser les performances. Au contraire, cette configuration dégrade systématiquement l'Accuracy (perte de 1.5% en Arkansas par rapport au Climat seul). Donner trop de variables (surtout des variables statiques peu utiles) "noie" les informations importantes dans un océan de données inutiles (noie le modèle dans du bruit). Ceci prouve l'importance du "feature selection" en Deep Learning.
3. La résolution des ambiguïtés :
L'apport des covariables cible précisément les faiblesses du modèle. En Californie, l'intégration du climat augmente le F1-Score Macro de 2,7% (passant de 0.7784 pour la Baseline à 0.8053). Puisque le F1-Score Macro accorde la même importance à toutes les classes (même les plus minoritaires ou difficiles), cette nette progression démontre que les données environnementales offrent un contexte non négligeable permettant au réseau de distinguer les cultures aux signatures spectrales quasi identiques, là où l'utilisation des bandes satellitaires seules atteignait ses limites.

Conclusion :

Les expériences menées démontrent que l'intégration de la dynamique temporelle (Climat) doit primer sur les variables spatiales brutes (Sol/Relief). L'optimisation des modèles de cartographie agricole ne passe pas par l'accumulation aveugle de données, mais par une sélection privilégiant la synergie entre le rythme des saisons et les mécanismes d'attention temporelle du réseau.

Partie 3 : Proposition d’architecture améliorée

Dans cette troisième partie, nous cherchons à concevoir une architecture exploitant le dataset enrichi avec covariables environnementales constitué en Partie 2. L’objectif est d’aller au-delà de la simple reproduction de MCTNet en proposant une modification architecturale ciblée.

3.1 Problématique : le salt-and-pepper noise dans MCTNet

3.1.1 Définition et origine du bruit

Le modèle MCTNet classique adopte une approche de **classification pixel-wise indépendante** : chaque pixel est traité de manière isolée, sans aucune connaissance du contexte de ses voisins. Si cette conception simplifie l’entraînement, elle engendre un problème structurel connu sous le nom de **salt-and-pepper noise** [4]; un artefact visuel caractéristique des cartes de classification pixel-wise où des pixels isolés reçoivent une étiquette incohérente avec leur environnement immédiat.

3.1.2 Mécanismes générateurs du bruit

trois mécanismes distincts contribuent à l’apparition de ce bruit dans les paysages agricoles hétérogènes tels que la Californie :

1. **Absence de contexte spatial local.** Le modèle ne dispose d’aucun accès aux pixels voisins lors de sa décision. Il ne peut donc pas exploiter l’autocorrélation spatiale naturellement présente dans les données agricoles : une parcelle de coton, par exemple, est quasi certainement entourée d’autres pixels de coton. Cette information de continuité est totalement ignorée par une architecture pixel-wise.
2. **Ambiguïté spectrale des pixels de bordure.** Les pixels situés à la frontière entre deux classes présentent un signal spectral mélangé (effets de mixité spectrale, *mixed pixels*). Sans contexte, ces pixels sont classés sur la seule base de leur signature temporelle, souvent insuffisante pour les discriminer.
3. **Sensibilité aux artefacts résiduels.** Des nuages partiellement filtrés, des ombres ou des anomalies radiométriques ponctuelles peuvent dégrader la signature spectro-temporelle d’un pixel isolé. Sans régularisation, ces artefacts se propagent directement dans la carte de classification finale.

Illustration du phénomène. Considérons une région sur laquelle MCTNet opère :

C	C	C	C	C	
C	S	C	C	C	C = Cotton
C	C	C	C	C	S = Soybean : pixel isolé
C	C	C	C	C	

Le pixel S, entouré de huit voisins classifiés C, constitue un artefact typique. Le modèle l’a classifié indépendamment, sans “voir” que l’ensemble de son voisinage géographique appartient à la classe coton. Dans une carte de classification réelle, plusieurs pixels aberrants dégradent la lisibilité et les métriques de précision, particulièrement en Californie.

3.2 Axe d’amélioration : régularisation par Batch-KNN

3.2.1 Positionnement de la solution

Face à cette limitation, deux grandes familles d’approches existent :

- Les approches **patch-based** [3] qui fournissent explicitement au modèle un voisinage spatial en entrée (fenêtre de $n \times n$ pixels). Elles résolvent directement le problème mais multiplient le coût computationnel et nécessitent une refonte complète du pipeline de données.
- Les approches de **post-traitement spatial** [1] (filtre médian, morphologie mathématique) appliquées sur la carte de prédiction finale. Simples à mettre en œuvre, elles opèrent cependant après la décision du modèle et n’influencent pas la qualité des représentations apprises.

Notre proposition emprunte une troisième voie : une **régularisation dans l’espace des représentations latentes**, intercalée entre l’encodeur MCTNet et le classifieur linéaire final. Cette approche présente l’avantage de stabiliser les représentations *avant* la décision, sans modifier l’architecture d’entrée ni le post-traitement.

3.2.2 Description de l’architecture MCTNet + Batch-KNN

L’architecture proposée est une extension directe de MCTNet dans laquelle un bloc de **lissage Batch-KNN** est inséré entre le pooling global et le classifieur linéaire. La classification demeure **pixel-wise** : chaque pixel produit toujours sa propre prédiction finale. Le Batch-KNN n’est pas implémenté comme un classifieur ; il opère exclusivement sur les vecteurs latents et en retourne une version lissée de même dimension.

Positionnement dans le flux.

Entrée (B, 36, 18) $\rightarrow$ input_proj $\rightarrow$ ALPE $\rightarrow$ CTFusion $\times 3 \rightarrow$
GlobalMaxPool $\rightarrow$ BatchKNN $\rightarrow$ Linear(80 $\rightarrow$ C) $\rightarrow$ Softmax

La sortie du `GlobalMaxPool` est un vecteur de dimension 80. Le Batch-KNN reçoit ce vecteur, l'enrichit par lissage, et retourne un vecteur enrichi de même dimension. Le classifieur opère ensuite sur ce dernier.

3.2.3 Fonctionnement du bloc Batch-KNN

Pour un batch de B pixels, le bloc Batch-KNN effectue les opérations suivantes sur les représentations latentes $\mathbf{Z} \in \mathbb{R}^{B \times 80}$ issues du `GlobalMaxPool` :

Algorithm 9 Batch-KNN Smoothing

Require: $\mathbf{Z} \in \mathbb{R}^{B \times 80}$

Require: $K = 8$

Ensure: $\tilde{\mathbf{Z}} \in \mathbb{R}^{B \times 80}$

```

1: si  $B \leq K$  alors
2:   retourner  $\mathbf{Z}$ 
3: fin si
4:  $\mathbf{D} \leftarrow \text{CDIST}(\mathbf{Z}, \mathbf{Z})$  ▷ distances euclidiennes ( $B \times B$ )
5: pour  $i = 1$  à  $B$  faire
6:    $\mathcal{N}(i) \leftarrow \text{TopK}(\mathbf{D}[i, :], K, \text{largest}=\text{False})$  ▷  $K$  indices les plus proches
7:    $\mathbf{Z}_{\mathcal{N}(i)} \leftarrow \mathbf{Z}[\mathcal{N}(i)]$  ▷ vecteurs voisins  $\in \mathbb{R}^{K \times 80}$ 
8:    $\tilde{\mathbf{z}}_i \leftarrow \frac{1}{K} \sum_{j \in \mathcal{N}(i)} \mathbf{z}_j$  ▷ moyenne des  $K$  voisins
9: fin pour
10: retourner  $\tilde{\mathbf{Z}} = [\tilde{\mathbf{z}}_1, \dots, \tilde{\mathbf{z}}_B]$  ▷ transmis au classifieur linéaire

```

Notre Batch-KNN ne connaît donc pas les coordonnées spatiales ; il opère sur les voisins *sémantiques* dans l'espace latent.

Il s'agit donc d'une **régularisation sémantique** : elle réduit la variance des représentations aberrantes (pixels bruités etc). Le bruit salt-and-pepper spatial est atténué *indirectement*, par stabilisation des décisions du classifieur, et non par exploitation directe de la contiguïté géographique.

L'intégration des variables climatiques temporelles à l'entrée du modèle pourrait renforcer la pertinence de ce lissage sémantique. Deux pixels géographiquement proches partagent généralement des conditions climatiques similaires ; ils produisent donc des représentations latentes proches. Le voisinage Batch-KNN capturera ainsi une forme de **proximité géo-climatique implicite**, sans nécessiter l'accès explicite aux coordonnées spatiales.

Exemple : Lissage du Pixel 0

ÉTAPE 1 : Extraire distances du Pixel 0

distances[0, :] = [0.0, 2.3, 1.8, 5.2, 3.1, 4.0, 2.1, 1.5, ...]
indices = [0, 1, 2, 3, 4, 5, 6, 7, ...]

ÉTAPE 2 : Trier et garder les 8 plus petites

distances_triées = [0.0, 1.5, 1.8, 2.1, 2.3, 3.1, 4.0, 5.2]
indices_triés = [0, 7, 2, 6, 1, 4, 5, 3]

Top-8 voisins du Pixel 0 :

[0, 7, 2, 6, 1, 4, 5, 3]

ÉTAPE 3 : Extraire les features des 8 voisins

Voisin 0 (Pixel 0) : [0.23, -0.45, 0.89, 0.12, -0.34, 0.67, ...]
Voisin 1 (Pixel 7) : [0.24, -0.44, 0.88, 0.13, -0.33, 0.66, ...]
Voisin 2 (Pixel 2) : [0.21, -0.48, 0.91, 0.10, -0.36, 0.65, ...]
Voisin 3 (Pixel 6) : [0.22, -0.47, 0.90, 0.13, -0.33, 0.68, ...]
Voisin 4 (Pixel 1) : [0.25, -0.43, 0.87, 0.15, -0.31, 0.69, ...]
Voisin 5 (Pixel 4) : [0.20, -0.49, 0.92, 0.11, -0.37, 0.64, ...]
Voisin 6 (Pixel 5) : [0.26, -0.42, 0.86, 0.16, -0.30, 0.70, ...]
Voisin 7 (Pixel 3) : [0.19, -0.50, 0.93, 0.09, -0.39, 0.63, ...]

ÉTAPE 4 : Calculer la moyenne pour chaque dimension

Dimension 0

$$\text{Moyenne} = \frac{0.23 + 0.24 + 0.21 + 0.22 + 0.25 + 0.20 + \dots}{8} = \frac{1.80}{8} = 0.225$$

... et ainsi de suite pour les 80 dimensions.

RÉSULTAT FINAL

features_lissées[0, :] = [0.225, -0.460, 0.895, 0.119, -0.341, 0.665, ...]

INTERPRÉTATION

- Le pixel 0 avait :

[0.23, -0.45, 0.89, ...]

- Après lissage :

[0.225, -0.460, 0.895, ...]

- Changement faible : les pixels voisins étaient similaires.
- Effet obtenu : moins de bruit et davantage de cohérence spatiale.

3.3 Architecture complète : composantes et modifications

3.3.1 Composantes héritées de MCTNet (Parties 1 et 2)

Les composantes suivantes sont reprises de MCTNet avec les adaptations dimensionnelles nécessaires à l'intégration des covariables :

- **Blocs CTFusionBlock (CNN-Transformer).**
- **Encodage de position adaptatif ALPE.**
- **AdaptiveMaxPool1d(1).** Après les trois blocs CTFusion, un pooling adaptatif comprime la dimension temporelle résiduelle en un vecteur unique $(B, 80)$, agrégeant l'information temporelle la plus saillante.

3.3.2 Modifications apportées en Partie 3

Par rapport à la version MCTNet de la Partie 1, deux modifications architecturales sont introduites :

1. **Couche de projection en entrée $\text{Linear}(F \rightarrow d)$.** Le passage de 10 à $F \in \{10, 13, 18\}$ features selon la configuration de covariables nécessite une projection dans l'espace du modèle ($d_{\text{model}} = 30$) avant l'ALPE et les blocs CTFusion. Sans cette projection, les dimensions seraient incompatibles avec les têtes d'attention.
2. **Bloc Batch-KNN** inséré entre le GlobalMaxPool et le classifieur. Il constitue l'innovation principale de cette partie, décrite en détail à la section précédente.
3. **Taille de batch : 64** (contre 32 en Partie 1). Un batch plus grand est nécessaire pour que le Batch-KNN dispose d'un voisinage suffisamment diversifié et représentatif. Avec $K = 8$ voisins et `batch_size=64`.

3.4 Résultats et analyse

3.4.1 Configurations évaluées

Nous avons évalué trois configurations d'entrée afin de mesurer la contribution respective de chaque groupe de covariables sur l'architecture MCTNet + Batch-KNN :

Table 3.1: Description des trois configurations d'entrée évaluées

Version	Covariables utilisées	Dimension F	Features
V1	Spectral + climat + statiques	18	$10 + 3 + 5$
V2	Spectral + climat	13	$10 + 3$
V3	Spectral uniquement	10	10

,

Version 1 : Covariables complètes (bandes spectrales + climat + statiques)

La première version utilise l'ensemble des covariables environnementales disponibles : 10 bandes spectrales, 3 variables climatiques temporelles et 5 variables statiques de sol et topographie, soit une dimension d'entrée de 18 features par pas de temps. (retrouvez le détail des graphes dans l'annexe)

Version 3 : Bandes spectrales uniquement

La troisième version utilise exclusivement les 10 bandes spectrales, sans aucune information climatique temporelle ni statique, réduisant la dimension d'entrée à 10 features par pas de temps. (retrouvez le détail des graphes dans l'annexe)

Version 2 : Bandes spectrales + variables climatiques (sans statiques)

La deuxième version exclut les variables statiques de sol et topographie et conserve uniquement les 10 bandes spectrales et les 3 variables climatiques temporelles, réduisant la dimension d'entrée à 13 features par pas de temps.

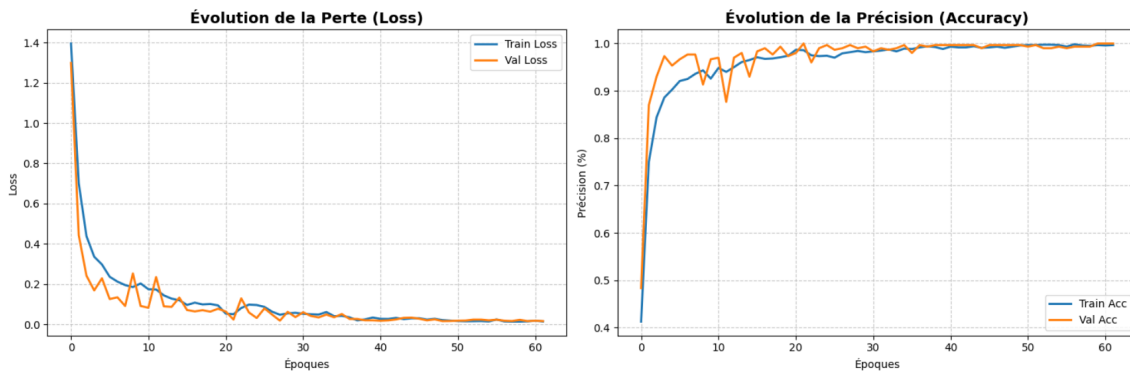


Figure 3.1: Courbes d'évaluation de la perte / précision pour MCTNet + Batch-KNN (Version 2) sur Arkansas.

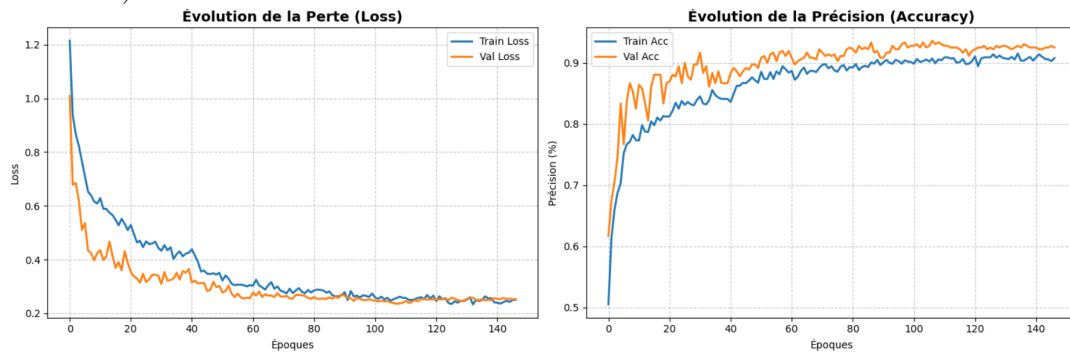


Figure 3.2: Courbes d'évaluation de la perte / précision pour MCTNet + Batch-KNN (Version 2) sur Californie

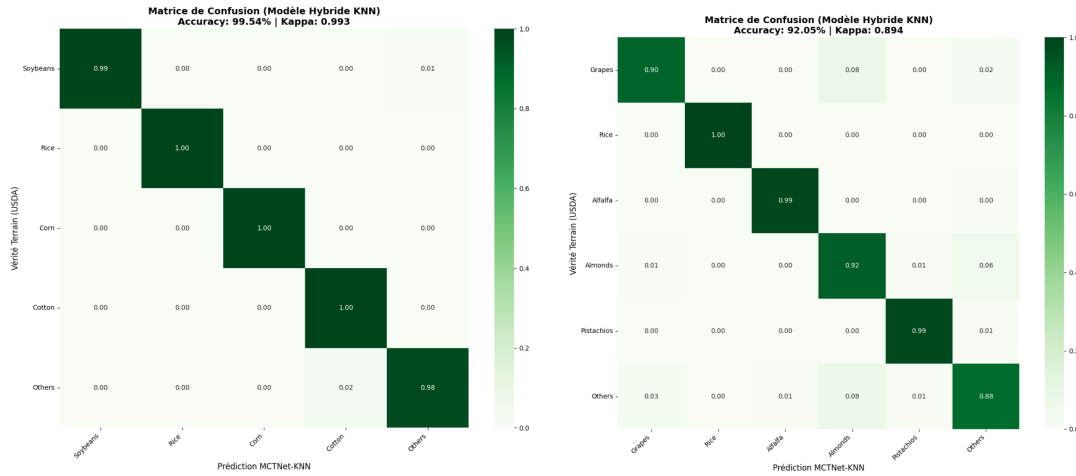


Figure 3.3: Matrices de confusion pour MCTNet + Batch-KNN (Version 2).

3.4.2 Résultats quantitatifs

Table 3.2: Comparaison des métriques pour les trois versions sur Arkansas et Californie (meilleurs résultats sur 10 runs)

Version	Arkansas			Californie		
	OA	Kappa	F1	OA	Kappa	F1
V1 — spec + clim + stat	0.9832	0.9743	0.9651	0.8984	0.8650	0.8619
V2 — spec + clim	0.9954	0.9929	0.9865	0.9205	0.8938	0.9025
V3 — spec	0.9936	0.9902	0.9822	0.8935	0.8580	0.8652

3.4.3 Analyse

La Version 2 est optimale sur les deux datasets c'est donc la version retenue dans notre code. L'ajout des seules variables climatiques temporelles (température, précipitation, point de rosée) à la signature spectrale produit le meilleur modèle : +1.2 points d'OA sur Arkansas et +2.2 points sur Californie par rapport à Sentinel-2 seul (Version 3). Ce résultat confirme l'hypothèse posée; les covariables climatiques apportent une information discriminante forte, directement liée à la phénologie des cultures, et structurent l'espace latent de manière à rendre le lissage Batch-KNN particulièrement cohérent.

Les variables statiques dégradent les performances (V1 < V2). Conformément aux observations de la Partie 2, l'ajout des variables de sol et de topographie (Version 1) diminue les performances par rapport à la Version 2 sur les deux zones d'étude, et ce de manière consistante.

La Californie demeure le cas le plus complexe. Sur l'ensemble des configurations, la Californie présente des OA systématiquement inférieures à celles d'Arkansas, reflétant la plus grande hétérogénéité de ce dataset. Mais c'est précisément sur la Californie que le gain apporté par les covariables climatiques est le plus prononcé (+2.2 points, V2 vs V3).

Bibliography

- [1] M. Durham. “The Use of a Median Filter to Improve Classification Accuracy”. In: *Geoscience and Remote Sensing Symposium Proceedings, IGARSS '98*. Vol. 2. 1998, pp. 1134–1136.
- [2] Ali Hassani. *Escaping the Big Data Paradigm with Compact Transformers*. 2021. URL: <https://arxiv.org/abs/2104.05704> (visited on 2026-05-06).
- [3] Nataliia Kussul et al. “Parcel-based crop classification in Ukraine using Landsat-8 data and Sentinel-1A data”. In: *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 10.6 (2017), pp. 2501–2508.
- [4] Zhaoxin Li et al. “A Lightweight CNN-Transformer Network for Pixel-Based Crop Mapping Using Time-Series Sentinel-2 Imagery”. In: *Remote Sensing* 15.3 (2023), p. 837. DOI: 10.3390/rs15030837.